

Q1. Consider that you are owning a supermarket mall and through membership cards, you have some basic data about your customers like Customer ID, age, gender, annual income and spending score. For the above scenario, the Problem Statement was You want to understand the customers who can easily converge [Target Customers] so that the data can be given to the marketing team and plan the strategy accordingly. For the above scenario prepare a dataset and perform Clustering Analysis to segment the customers in the Mall. There are clearly Five segments of Customers based on their Annual Income and Spending Score namely Usual Customers, Priority Customers, Senior Citizen Target Customers, and Young Target Customers. Sample data.

CODE:

@relation mall_customers

@attribute CustomerID numeric

@attribute Gender {Male, Female}

@attribute Age numeric

@attribute Annual_Income numeric

@attribute Spending_Score numeric

@data

1, Male, 19, 15, 39

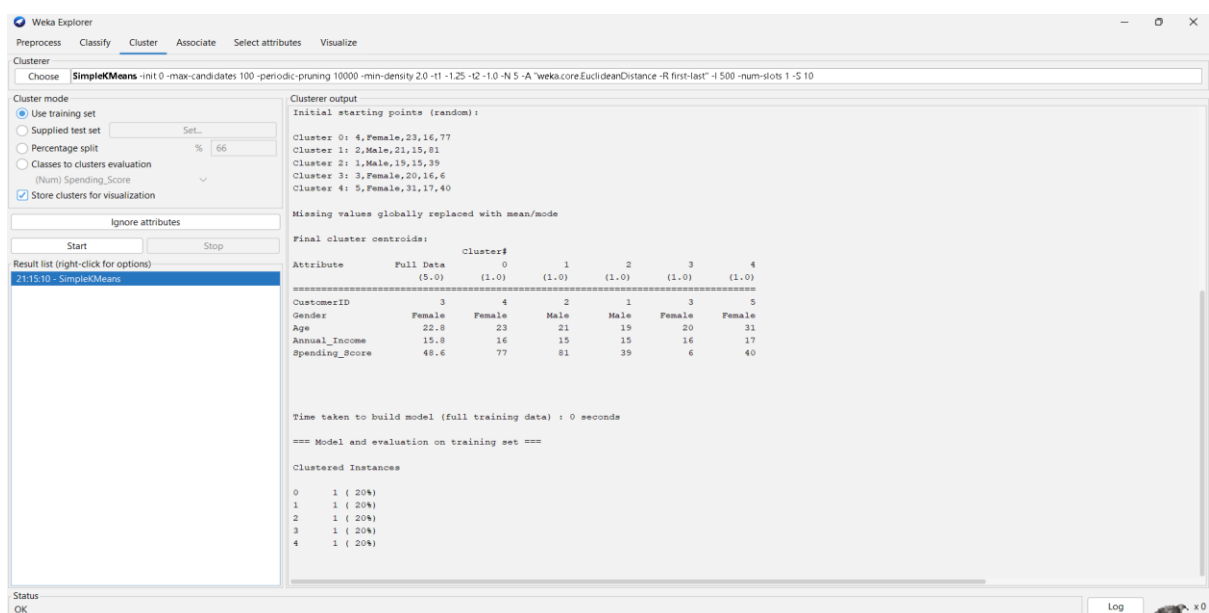
2, Male, 21, 15, 81

3, Female, 20, 16, 6

4, Female, 23, 16, 77

5, Female, 31, 17, 40

OUTPUT:



Weka Explorer

Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer: Choose **SimpleKMeans** -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 5 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10

Cluster mode:
☒ Use training set
☐ Supplied test set Set...
☐ Percentage split % 66
☐ Classes to clusters evaluation (Num) Spending_Score
☒ Store clusters for visualization

Ignore attributes: []

Start Stop

Result list (right-click for options):
 21:15:10 - SimpleKMeans

Cluster output

Initial starting points (random):
 Cluster 0: 4, Female, 23, 16, 77
 Cluster 1: 2, Male, 21, 15, 81
 Cluster 2: 1, Male, 19, 15, 39
 Cluster 3: 3, Female, 20, 16, 6
 Cluster 4: 5, Female, 31, 17, 40

Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute	Full Data (5.0)	Cluster# 0 (1.0)	Cluster# 1 (1.0)	Cluster# 2 (1.0)	Cluster# 3 (1.0)	Cluster# 4 (1.0)
CustomerID	3	4	2	1	3	5
Gender	Female	Female	Male	Male	Female	Female
Age	22.8	23	21	19	20	31
Annual_Income	15.8	16	15	15	16	17
Spending_Score	48.6	77	81	39	6	40

Time taken to build model (full training data) : 0 seconds

==== Model and evaluation on training set ====

Clustered Instances

0	1 (20%)
1	1 (20%)
2	1 (20%)
3	1 (20%)
4	1 (20%)

Status: OK

Q2. Create the following dataset using CSV file format. To perform cluster analysis using K-Means in WEKA. To change the cluster size and plot the graph and illustrate the visualization of cluster.

CODE:

EmployeeID,Gender,Age,Salary,Credit

111,Male,28,150000,39

222,Male,25,150000,27

333,Female,26,160000,42

444,Female,25,160000,40

555,Female,30,170000,64

666,Male,29,200000,72

OUTPUT:

Weka Explorer

Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer: Choose **SimpleKMeans** -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A 'weka.core.EuclideanDistance -R first-last' -I 500 -num-slots 1 -S 10

Cluster mode:

- ☒ Use training set
- ☐ Supplied test set
- ☐ Percentage split
- ☐ Classes to clusters evaluation
- ☒ Store clusters for visualization

Ignore attributes: []

Start Stop

Result list (right-click for options):

- 21:15:10 - SimpleKMeans
- 21:23:33 - SimpleKMeans**

Cluster output:

KMeans

Number of iterations: 2

Within cluster sum of squared errors: 2.9514403292181073

Initial starting points (random):

Cluster 0: 444,Female,25,160000,40

Cluster 1: 111,Male,28,150000,39

Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute	Full Data	Cluster# 0	Cluster# 1
EmployeeID	389.5	444	333
Gender	Male	Female	Male
Age	27.1667	25	27.3333
Salary	165000	163333.3333	166666.6667
Credit	47.3333	40.6667	46

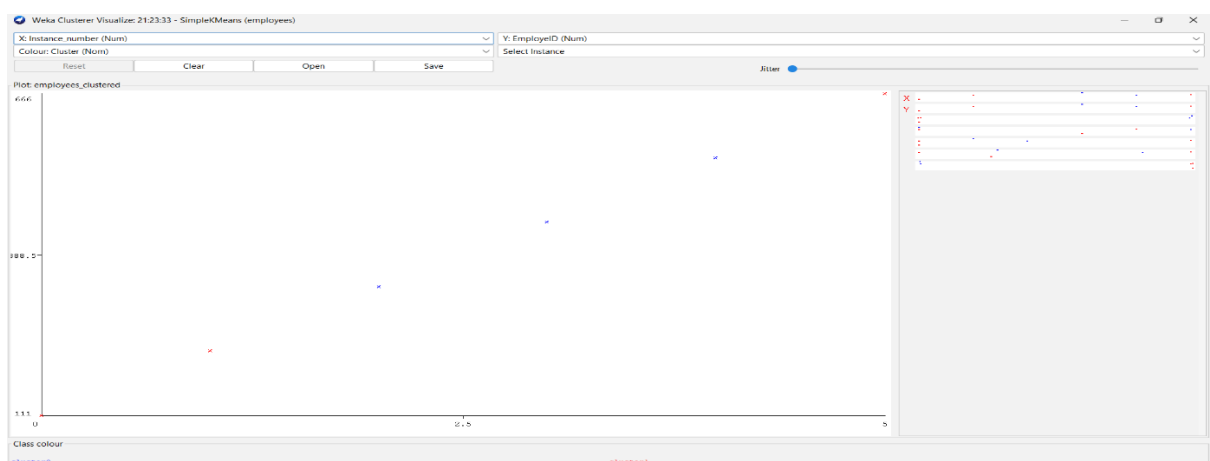
Time taken to build model (full training data) : 0 seconds

=== Model and evaluation on training set ===

Clustered Instances:

Cluster	Count	Percentage
0	3	50%
1	3	50%

Status: OK



Q3.

CODE:

@relation weather

@attribute outlook {sunny, overcast, rainy}

@attribute temperature {hot, mild, cool}

@attribute humidity {high, normal}

@attribute windy {TRUE, FALSE}

@attribute play {yes, no}

@data

sunny,hot,high,FALSE,no

sunny,hot,high,TRUE,no

overcast,hot,high,FALSE,yes

rainy,mild,high,FALSE,yes

rainy,cool,normal,FALSE,yes

rainy,cool,normal,TRUE,no

OUTPUT:

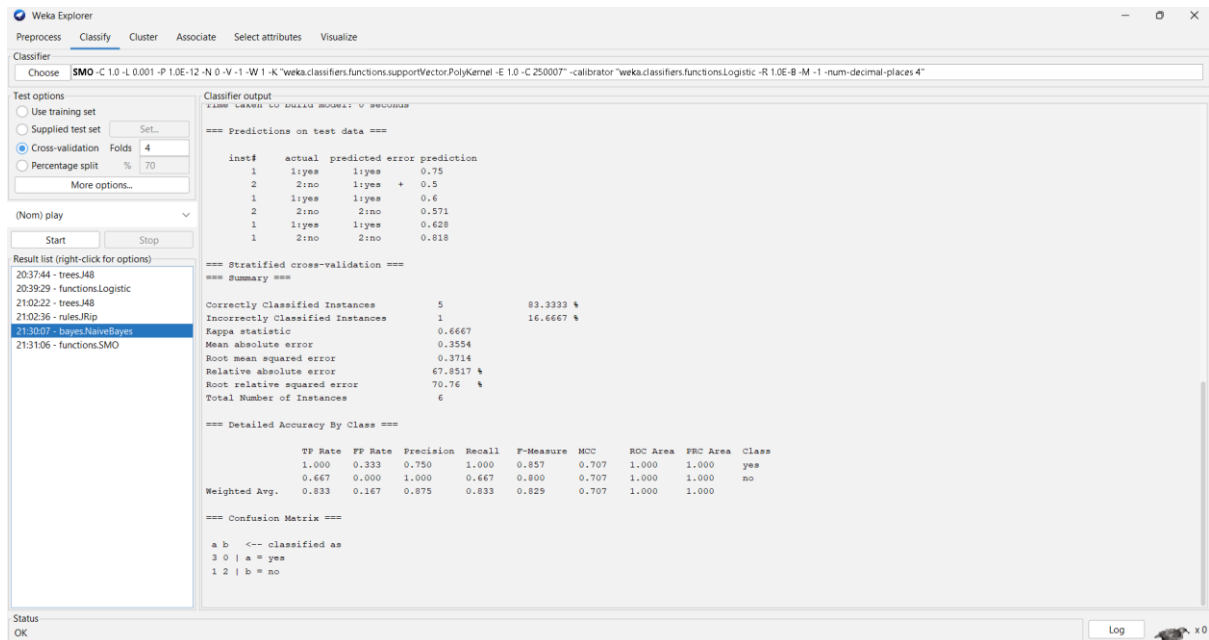
The screenshot shows the Weka Explorer interface with the 'Classify' tab selected. The classifier chosen is 'SMO - C 1.0 - L 0.001 - P 1.0E-12 - N 0 - V -1 - W 1 - K "weka.classifiers.functions.supportVector.PolyKernel - E 1.0 - C 250007" - calibrator "weka.classifiers.functions.Logistic - R 1.0E-8 - M -1 - num-decimal-places 4"'. The test options are set to 'Cross-validation' with 'Folds' set to 4. The result list on the left shows several classifiers, with '213106 - functions.SMO' selected. The classifier output is displayed in the main pane, showing predictions on test data, stratified cross-validation summary, detailed accuracy by class, and a confusion matrix.

```
=== Predictions on test data ===
inst#  actual  predicted error prediction
1      1:yes   1:yes      1
2      2:no    1:yes      + 1
1      1:yes   1:yes      1
2      2:no    1:yes      + 1
1      1:yes   1:yes      1
2      2:no    2:no      1

=== Stratified cross-validation ===
=== Summary ===
Correctly Classified Instances      4      66.6667 %
Incorrectly Classified Instances    2      33.3333 %
Kappa statistic                    0.3333
Mean absolute error                 0.3333
Root mean squared error             0.5774
Relative absolute error             63.6364 %
Root relative squared error        109.5964 %
Total Number of Instances          6

=== Detailed Accuracy By Class ===
               TP Rate  FP Rate  Precision  Recall  F-Measure  MCC      ROC Area  PRC Area  Class
               ----
1.000  0.667  0.600  1.000  0.750  0.447  0.667  0.600  yes
0.333  0.000  1.000  0.333  0.500  0.447  0.667  0.667  no
Weighted Avg.  0.667  0.333  0.800  0.667  0.625  0.447  0.667  0.633

=== Confusion Matrix ===
a b  <-- classified as
3 0 | a = yes
2 1 | b = no
```



Q4. The following list of persons with vegetarian or not details given in the table. How will you find out how many of them are vegetarian and how many of them are non-vegetarian?

Which type of the person total count is greater value?

CODE:

@relation vegetarian_dataset

@attribute Person string

@attribute Vegetarian {yes, no}

@data

Gopu, yes

Babu, yes

Baby, yes

Gopal, no

Krishna, yes

Jai, no

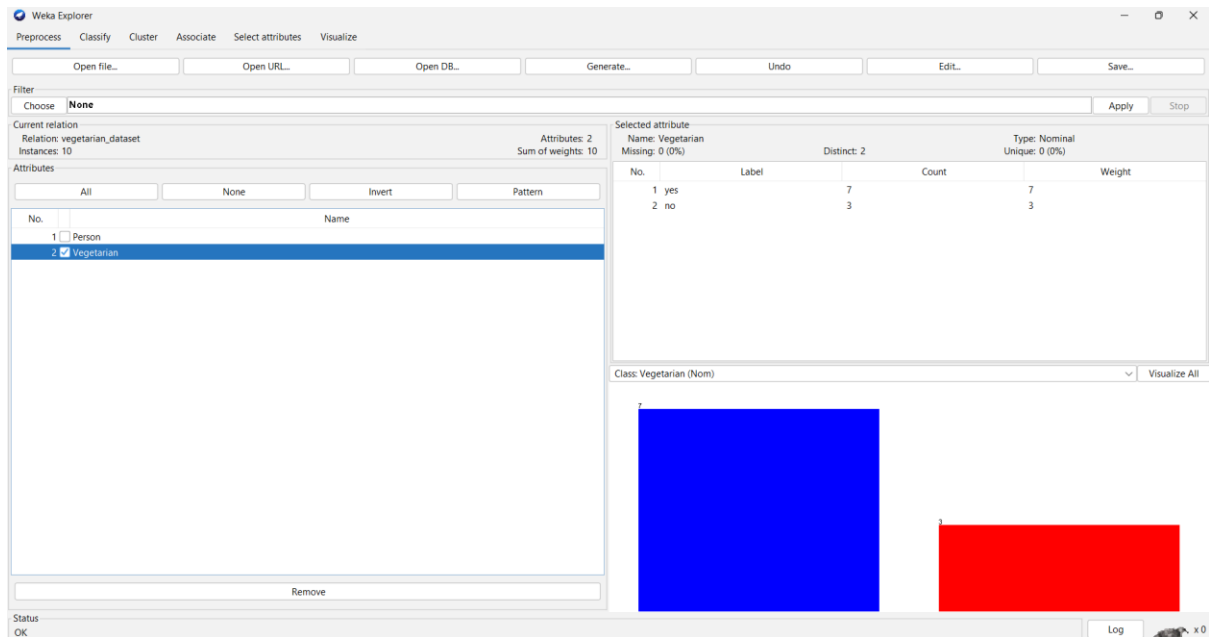
Dev, no

Malini, yes

Hema, yes

Anu, yes

OUTPUT:



Q5. The following table would be plotted as (x,y) points, with the first column being the x values as number of mobile phones sold and the second column being the y values as money.

To use the scatter plot for how many mobile phones sold.

CODE:

x,y

4,12

1,5

5,13

7,19

10,31

2,7

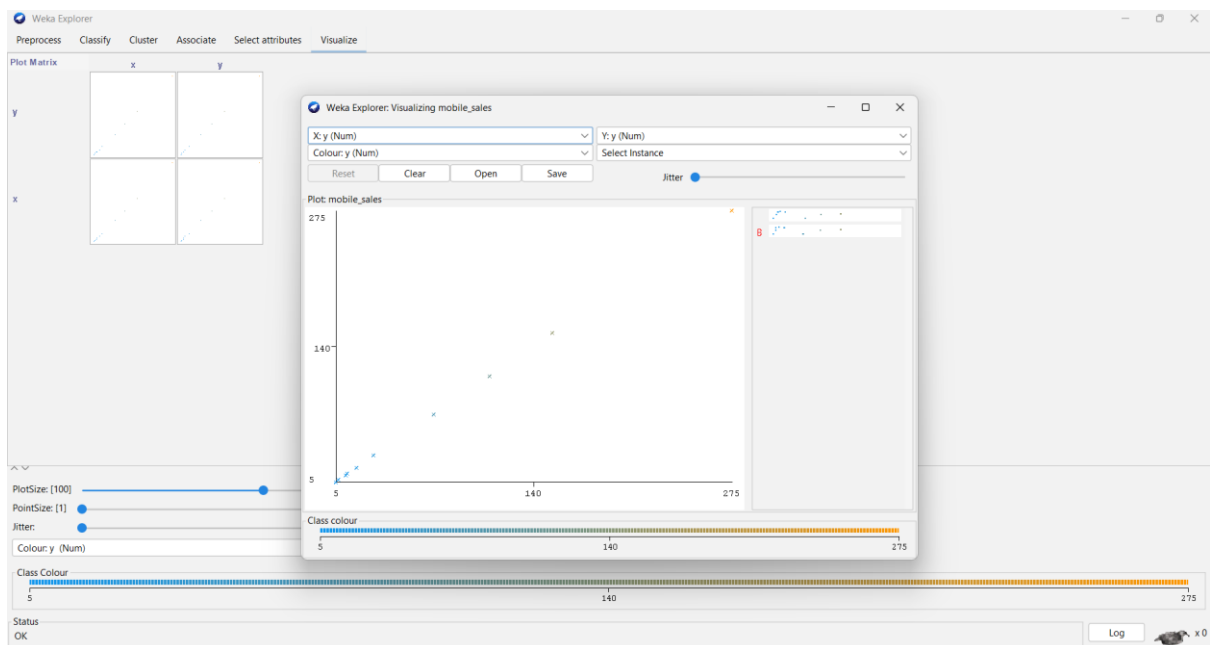
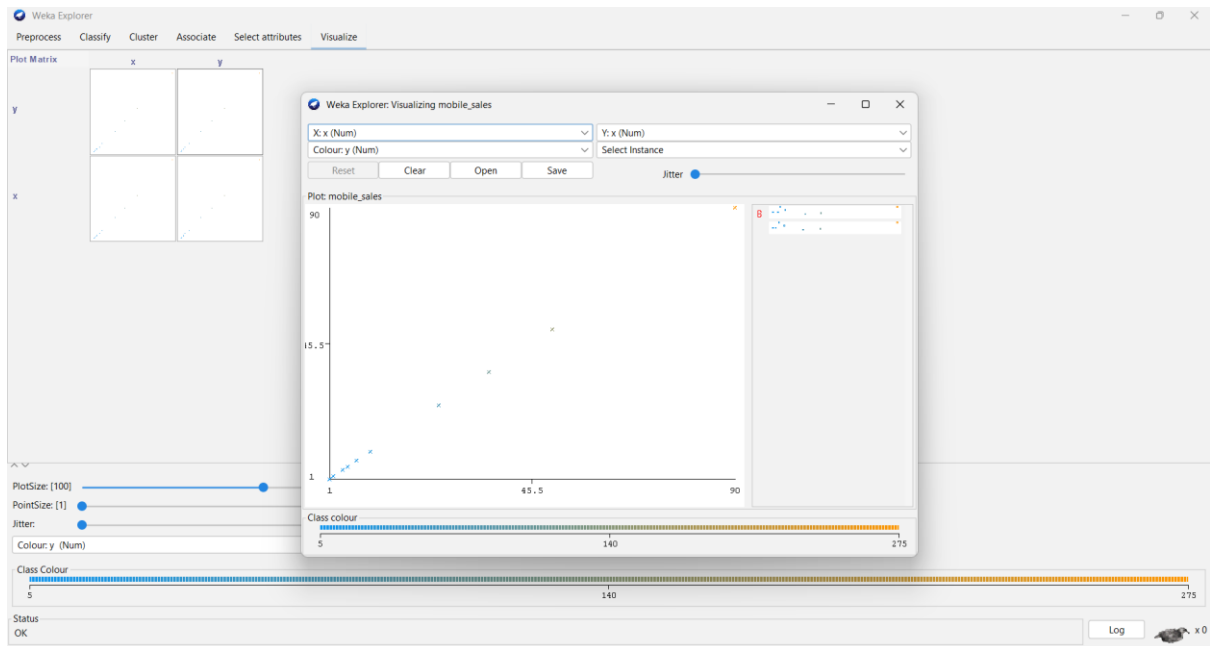
50,153

25,72

90,275

36,110

OUTPUT:



Q6. Generate rules using FP growth algorithm using the given dataset which has the following transactions with items purchased: Consider the values as support=50% and confidence=75%.

CODE:

@relation supermarket_transactions

@attribute Bread {t, f}

@attribute Cheese {t, f}

@attribute Egg {t, f}

@attribute Juice {t, f}

@attribute Milk {t, f}

@attribute Yogurt {t, f}

@data

t, t, t, t, f, f

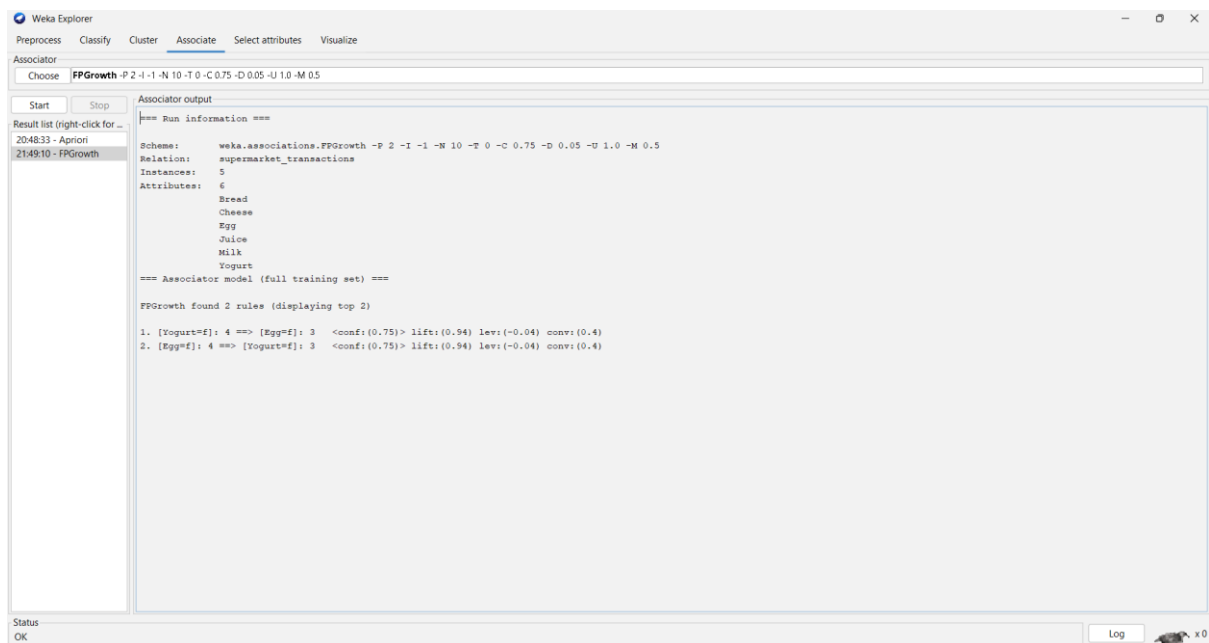
t, t, f, t, f, f

t, f, f, f, t, t

t, f, f, t, t, f

f, t, f, t, t, f

OUTPUT:



Q7. Prediction of Diabetes Data using Decision tree classifier in WEKA. Compare it with Support Vector Machine classifier. Show the result accuracy and F1 measure calculation .Plot the graph and explain the summary of results.

CODE:

@relation pima_diabetes_numeric

@attribute 'preg' numeric

@attribute 'plas' numeric

@attribute 'pres' numeric

@attribute 'skin' numeric

@attribute 'insu' numeric

@attribute 'mass' numeric

@attribute 'pedi' numeric

@attribute 'age' numeric

@attribute 'class' numeric

@data

6,148,72,35,0,33.6,0.627,50,1

1,85,66,29,0,26.6,0.351,31,0

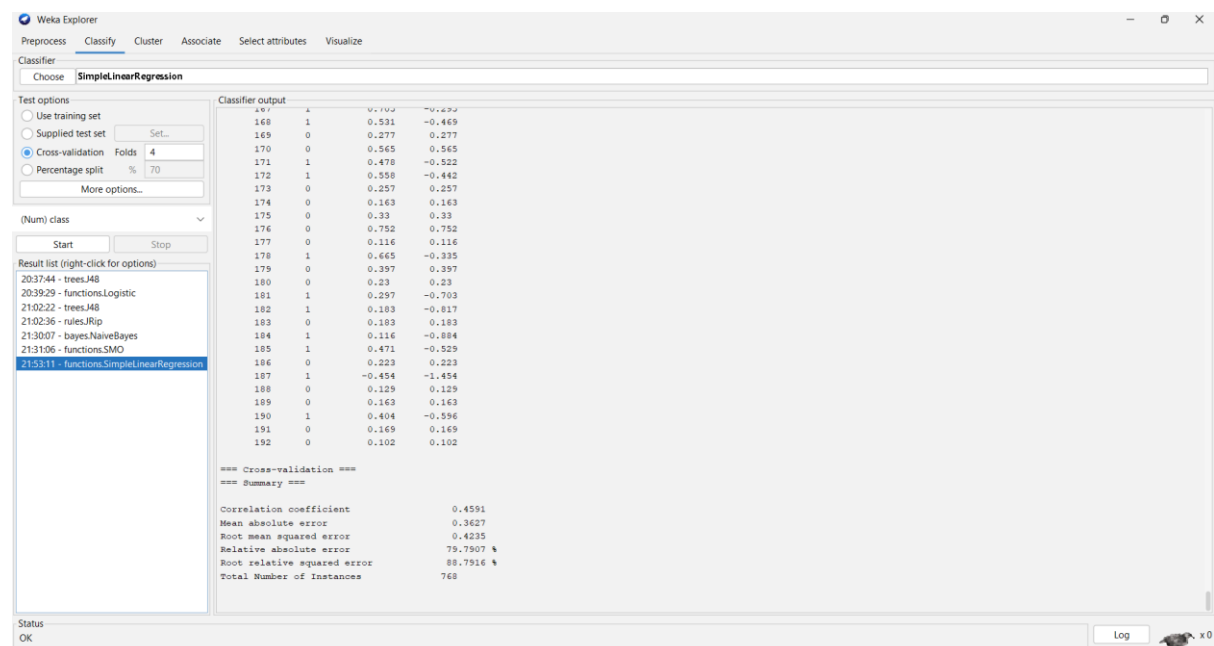
8,183,64,0,0,23.3,0.672,32,1

1,89,66,23,94,28.1,0.167,21,0

0,137,40,35,168,43.1,2.288,33,1

5,116,74,0,0,25.6,0.201,30,0

OUTPUT:



Q8. Implement of the R script using marks scored by a student in his model exam has been sorted as follows: 55, 60, 71, 63, 55, 65, 50, 55, 58, 59, 61, 63, 65, 67, 71, 72, 75. Partition them into three bins by each of the following methods. Plot the data points using histogram.

(a) equal-frequency (equi-depth) partitioning

(b) equal-width partitioning

(c) clustering

CODE:


```

# Given marks of the student in the model exam
marks <- c(55, 60, 71, 63, 55, 65, 50, 55, 58, 59, 61, 63, 65, 67, 71, 72, 75)

# Sort the data
sorted_marks <- sort(marks)
print("Sorted Marks:")
print(sorted_marks)

# -----
# (a) Equal-Frequency (Equi-Depth) Partitioning (3 bins)
# -----
n <- length(sorted_marks)
k <- 3
# Find quantiles to split into 3 equal-frequency bins
eq_freq_bins <- cut(sorted_marks,
                     breaks = quantile(sorted_marks, probs = seq(0, 1, length.out = k + 1)),
                     include.lowest = TRUE)
print("Equal-Frequency Bins:")
print(table(eq_freq_bins))

# -----
# (b) Equal-Width Partitioning (3 bins)
# -----
min_val <- min(sorted_marks)
max_val <- max(sorted_marks)
width <- (max_val - min_val) / k

eq_width_bins <- cut(sorted_marks,
                     breaks = seq(min_val, max_val, length.out = k + 1),
                     include.lowest = TRUE)

```

```

print("Equal-Width Bins:")
print(table(eq_width_bins))

# -----
# (c) Clustering (K-Means with k = 3)
# -----
# Reshape data for kmeans function
set.seed(42) # For reproducibility
kmeans_result <- kmeans(sorted_marks, centers = 3)
print("Clustering Assignments:")
print(kmeans_result$cluster)

# -----
# Plotting Histograms
# -----
par(mfrow = c(1, 3)) # Set layout for 3 plots

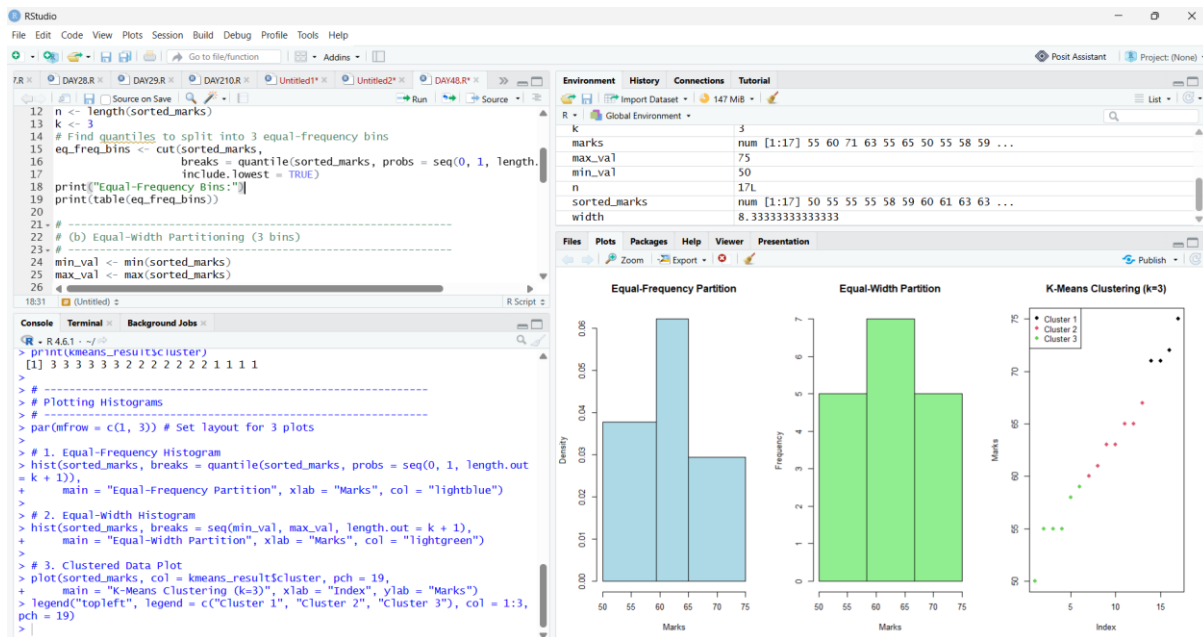
# 1. Equal-Frequency Histogram
hist(sorted_marks, breaks = quantile(sorted_marks, probs = seq(0, 1, length.out = k + 1)),
      main = "Equal-Frequency Partition", xlab = "Marks", col = "lightblue")

# 2. Equal-Width Histogram
hist(sorted_marks, breaks = seq(min_val, max_val, length.out = k + 1),
      main = "Equal-Width Partition", xlab = "Marks", col = "lightgreen")

# 3. Clustered Data Plot
plot(sorted_marks, col = kmeans_result$cluster, pch = 19,
      main = "K-Means Clustering (k=3)", xlab = "Index", ylab = "Marks")
legend("topleft", legend = c("Cluster 1", "Cluster 2", "Cluster 3"), col = 1:3, pch = 19)

```

OUTPUT:



Q9. The given are the strike-rates scored by a batsman in season 1 in different tournaments.
100, 70, 60, 90, 90

- min-max normalization by setting $\min = 0$ and $\max = 1$
- z-score normalization
- z-score normalization using the mean absolute deviation instead of standard deviation
- normalization by decimal scaling

CODE:

Given data

```
strike_rates <- c(100, 70, 60, 90, 90)
```

(a) Min-Max Normalization ($\min = 0$, $\max = 1$)

```
min_val <- min(strike_rates)
```

```
max_val <- max(strike_rates)
```

```
min_max_norm <- (strike_rates - min_val) / (max_val - min_val)
```

```
print("Min-Max Normalized:")
```

```
print(min_max_norm)
```

(b) Z-Score Normalization (using population standard deviation)

```

mean_val <- mean(strike_rates)

pop_sd <- sqrt(sum((strike_rates - mean_val)^2) / length(strike_rates))

z_score_norm <- (strike_rates - mean_val) / pop_sd

print("Z-Score Normalized:")

print(z_score_norm)

```

(c) Z-Score Normalization using Mean Absolute Deviation (MAD)

```

mad_val <- mean(abs(strike_rates - mean_val))

mad_norm <- (strike_rates - mean_val) / mad_val

print("Z-Score Normalized using MAD:")

print(mad_norm)

```

(d) Normalization by Decimal Scaling

```

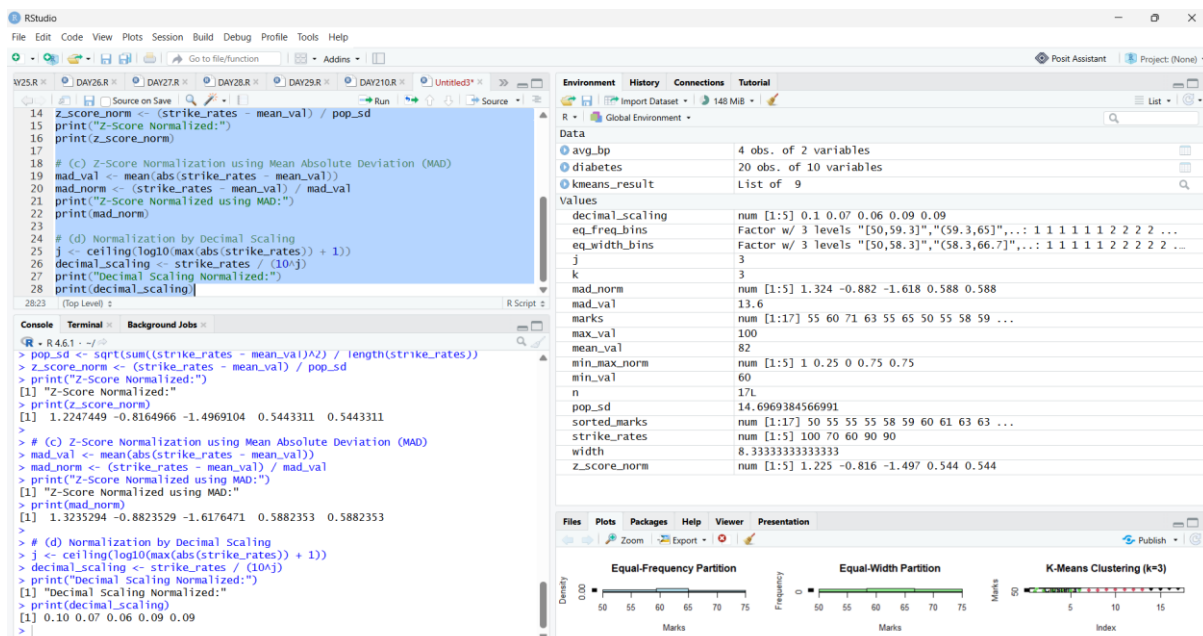
j <- ceiling(log10(max(abs(strike_rates)) + 1))

decimal_scaling <- strike_rates / (10^j)

print("Decimal Scaling Normalized:")

print(decimal_scaling)

```



Q10. Suppose some car is tested for the AvgSpeed and TotalTime data for 9 randomly selected car with the following result

- Calculate the standard deviation of AvgSpeed and TotalTime.
- Calculate the Variance of AvgSpeed and TotalTime for the above dataset.

CODE:

Given data vectors

```
avg_speed <- c(78, 81, 82, 74, 83, 82, 77, 80, 70)
```

```
total_time <- c(39, 37, 36, 42, 35, 36, 40, 38, 46)
```

(b) Calculate Variance (sample variance using n - 1 degrees of freedom)

```
var_avg_speed <- var(avg_speed)
```

```
var_total_time <- var(total_time)
```

```
print(paste("Variance of AvgSpeed:", round(var_avg_speed, 2)))
```

```
print(paste("Variance of TotalTime:", round(var_total_time, 2)))
```

(a) Calculate Standard Deviation

```
sd_avg_speed <- sd(avg_speed)
```

```
sd_total_time <- sd(total_time)
```

```
print(paste("Standard Deviation of AvgSpeed:", round(sd_avg_speed, 2)))
```

```
print(paste("Standard Deviation of TotalTime:", round(sd_total_time, 2)))
```

OUTPUT:

The screenshot shows the RStudio interface with the following R script in the editor:

```

3 total_time <- c(39, 37, 36, 42, 35, 36, 40, 38, 46)
4
5 # (b) Calculate Variance (sample variance using n - 1 degrees of freedom)
6 var_avg_speed <- var(avg_speed)
7 var_total_time <- var(total_time)
8
9 print(paste("Variance of AvgSpeed:", round(var_avg_speed, 2)))
10 print(paste("Variance of TotalTime:", round(var_total_time, 2)))
11
12 # (a) Calculate Standard Deviation
13 sd_avg_speed <- sd(avg_speed)
14 sd_total_time <- sd(total_time)
15
16 print(paste("Standard Deviation of AvgSpeed:", round(sd_avg_speed, 2)))
17 print(paste("Standard Deviation of TotalTime:", round(sd_total_time, 2)))
18

```

The console output shows the following results:

```

> avg_speed <- c(78, 81, 82, 74, 83, 82, 77, 80, 70)
> total_time <- c(39, 37, 36, 42, 35, 36, 40, 38, 46)
>
> # (b) Calculate Variance (sample variance using n - 1 degrees of freedom)
> var_avg_speed <- var(avg_speed)
> var_total_time <- var(total_time)
>
> print(paste("Variance of AvgSpeed:", round(var_avg_speed, 2)))
[1] "Variance of AvgSpeed: 18.53"
> print(paste("Variance of TotalTime:", round(var_total_time, 2)))
[1] "Variance of TotalTime: 12.19"
>
> # (a) Calculate Standard Deviation
> sd_avg_speed <- sd(avg_speed)
> sd_total_time <- sd(total_time)
>
> print(paste("Standard Deviation of AvgSpeed:", round(sd_avg_speed, 2)))
[1] "Standard Deviation of AvgSpeed: 4.3"
> print(paste("Standard Deviation of TotalTime:", round(sd_total_time, 2)))
[1] "Standard Deviation of TotalTime: 3.49"
>

```

The Environment pane on the right shows the following objects:

- avg_bp: 4 obs. of 2 variables
- diabetes: 20 obs. of 10 variables
- kmeans_result: List of 9
- avg_speed: num [1:9] 78 81 82 74 83 82 77 80 70
- decimal_scaling: num [1:5] 0.1 0.07 0.06 0.09 0.09
- eq_freq_bins: Factor w/ 3 levels "[50,59.3]", "(59.3,65]", "...: 1 1 1 1 1 2 2 2 ...
- eq_width_bins: Factor w/ 3 levels "[50,58.3]", "(58.3,66.7]", "...: 1 1 1 1 1 2 2 2 ...
- j: 3
- k: 3
- mad_norm: num [1:5] 1.324 -0.882 -1.618 0.588 0.588
- mad_val: 13.6
- marks: num [1:17] 55 60 71 63 55 65 50 55 58 59 ...
- max_val: 100
- mean_val: 82
- min_max_norm: num [1:5] 1 0.25 0 0.75 0.75
- min_val: 60
- n: 17L
- pop_sd: 14.6969384566991
- sd_avg_speed: 4.30439052338165
- sd_total_time: 3.49205447329283
- sorted_marks: num [1:17] 50 55 55 55 58 59 60 61 63 63 ...
- strike_rates: num [1:5] 100 70 60 90 90
- total_time: num [1:9] 39 37 36 42 35 36 40 38 46
- var_avg_speed: 18.5277777777778
- var_total_time: 12.1944444444444
- width: 8.33333333333333
- z_score_norm: num [1:5] 1.225 -0.816 -1.497 0.544 0.544